

Тема 4.4 Обмен данными с помощью модулей

Введение в библиотеку Requests

Requests — это простая и мощная библиотека для работы с HTTP-запросами в Python. Она обеспечивает удобный интерфейс для выполнения операций с REST API, загрузки данных с веб-страниц, а также отправки файлов и авторизации. Благодаря лаконичному синтаксису и широкому функционалу, Requests стала стандартом де-факто среди HTTP-клиентов Python.

Библиотека Requests была создана Кеннетом Рейтцем в 2011 году с целью сделать HTTP-запросы в Python максимально простыми и удобными. Она построена поверх urllib3 и предоставляет высокоуровневый API для работы с HTTP-протоколом.

Преимущества Requests по сравнению с urllib:

- более читаемый синтаксис — интуитивно понятные методы и параметры;
- автоматическая обработка куки и редиректов — не требует дополнительной настройки;
- удобная работа с JSON, заголовками, формами — встроенная поддержка популярных форматов;
- поддержка сессий, прокси, аутентификации — полный набор инструментов для работы с веб-сервисами;
- поддержка всех методов HTTP — GET, POST, PUT, DELETE, PATCH, HEAD, OPTIONS;
- автоматическое декодирование контента — интеллектуальная обработка кодировок;
- встроенная поддержка SSL — безопасные соединения по умолчанию.
-

Установка и подключение библиотеки

Установка с помощью pip:

```
pip install requests
```

Для обновления до последней версии:

```
pip install --upgrade requests
```

Импорт библиотеки:

```
import requests
```

Основные методы HTTP-запросов

Отправка GET-запросов

GET-запросы используются для получения данных с сервера:

```
response = requests.get('https://api.example.com/data')
```

```
print(response.status_code) # Код ответа
```

```
print(response.text) # Текст ответа
```

Получение JSON-данных:

```
response = requests.get('https://api.example.com/users')
```

```
data = response.json() # Автоматическое преобразование в словарь Python
print(data)
```

Отправка POST-запросов

POST-запросы используются для отправки данных на сервер:

```
payload = {'username': 'admin', 'password': '123'}
```

```
response = requests.post('https://api.example.com/login', data=payload)
```

Отправка JSON-данных:

```
user_data = {'name': 'John', 'email': 'john@example.com'}
```

```
response = requests.post('https://api.example.com/users', json=user_data)
```

Использование PUT, PATCH и DELETE

PUT - полное обновление ресурса

```
requests.put('https://api.example.com/user/1', json={'name': 'New Name',
'email': 'new@example.com'})
```

PATCH - частичное обновление ресурса

```
requests.patch('https://api.example.com/user/1', json={'active': True})
```

DELETE - удаление ресурса

```
requests.delete('https://api.example.com/user/1')
```

HEAD - получение только заголовков

```
response = requests.head('https://api.example.com/user/1')
```

```
print(response.headers)
```

Работа с параметрами запроса

Параметры URL можно передавать через словарь:

```
params = {'search': 'python', 'limit': 10, 'page': 1}
```

```
response = requests.get('https://api.example.com/search', params=params)
```

```
print(response.url) # Покажет полный URL с параметрами
```

Можно также передавать параметры в виде списка кортежей:

```
params = [('tag', 'python'), ('tag', 'web'), ('limit', 20)]
```

```
response = requests.get('https://api.example.com/posts', params=params)
```

Работа с заголовками и куки

Заголовки запроса

```
headers = {
```

```
    'Authorization': 'Bearer TOKEN123',
```

```
    'User-Agent': 'MyApp/1.0',
```

```
    'Content-Type': 'application/json'
```

```
}
```

```
response = requests.get('https://api.example.com/data', headers=headers)
```

Заголовки ответа

```
response = requests.get('https://api.example.com/data')
```

```
print(response.headers['Content-Type'])
```

```
print(response.headers.get('Server', 'Unknown'))
```

Работа с куки

```
# Отправка куки
cookies = {'sessionid': 'abc123', 'csrftoken': 'xyz789'}
response = requests.get('https://example.com', cookies=cookies)
# Получение куки из ответа
response = requests.get('https://example.com')
print(response.cookies)
for cookie in response.cookies:
    print(f'{cookie.name}: {cookie.value}')
```

Отправка и загрузка файлов

Отправка файлов

```
# Отправка одного файла
files = {'file': open('report.pdf', 'rb')}
response = requests.post('https://api.example.com/upload', files=files)
# Отправка нескольких файлов
files = {
    'file1': open('document.pdf', 'rb'),
    'file2': open('image.png', 'rb')
}
response = requests.post('https://api.example.com/upload', files=files)
# Отправка файла с дополнительными данными
files = {'file': (open('report.pdf', 'rb'), 'application/pdf')}
data = {'description': 'Monthly report'}
response = requests.post('https://api.example.com/upload', files=files,
data=data)
```

Загрузка файлов

```
# Загрузка файла в память
response = requests.get('https://example.com/file.zip')
with open('output.zip', 'wb') as f:
    f.write(response.content)
# Поточковая загрузка больших файлов
response = requests.get('https://example.com/large-file.zip', stream=True)
with open('large-file.zip', 'wb') as f:
    for chunk in response.iter_content(chunk_size=8192):
        f.write(chunk)
```

Обработка JSON и форм

Работа с JSON

```
# Отправка JSON
data = {'name': 'Alice', 'age': 30}
response = requests.post('https://api.example.com/users', json=data)
# Получение JSON
```

```

response = requests.get('https://api.example.com/users/1')
try:
    user_data = response.json()
    print(user_data['name'])
except ValueError:
    print("Ответ не содержит валидный JSON")
Отправка форм
# Форма с данными
form_data = {
    'username': 'user123',
    'password': 'secret',
    'remember': 'on'
}
response = requests.post('https://example.com/login', data=form_data)
# Форма с файлом
form_data = {'title': 'My Document'}
files = {'document': open('doc.pdf', 'rb')}
response = requests.post('https://example.com/upload', data=form_data,
files=files)

```

Работа с сессиями

Сессии позволяют сохранять состояние между запросами:

```

session = requests.Session()
# Настройка сессии
session.headers.update({'User-Agent': 'MyApp/1.0'})
session.auth = ('username', 'password')
# Выполнение запросов через сессию
session.get('https://example.com/login')
response = session.post('https://example.com/dashboard', data={'key': 'value'})
response = session.get('https://example.com/profile')
# Закрытие сессии
session.close()

```

Использование сессии в контекстном менеджере:

```

with requests.Session() as session:
    session.get('https://example.com/login')
    response = session.get('https://example.com/data')

```

Обработка исключений и ошибок

```

try:
    response = requests.get('https://api.example.com/data', timeout=5)
    response.raise_for_status() # Вызывает исключение для 4xx и 5xx
    data = response.json()

```

статусов

```
except requests.exceptions.HTTPError as e:
    print(f"HTTP ошибка: {e}")
except requests.exceptions.ConnectionError as e:
    print(f"Ошибка подключения: {e}")
except requests.exceptions.Timeout as e:
    print(f"Таймаут: {e}")
except requests.exceptions.RequestException as e:
    print(f"Общая ошибка запроса: {e}")
except ValueError as e:
    print(f"Ошибка парсинга JSON: {e}")
```

Установка таймаутов и повторных попыток

Таймауты

```
# Общий таймаут
response = requests.get('https://example.com', timeout=5)
# Раздельные таймауты для подключения и чтения
response = requests.get('https://example.com', timeout=(3, 10))
```

Повторные попытки

```
from requests.adapters import HTTPAdapter
from requests.packages.urllib3.util.retry import Retry
session = requests.Session()
# Настройка стратегии повторов
retry_strategy = Retry(
    total=3,          # Общее количество попыток
    backoff_factor=1, # Множитель для экспоненциального backoff
    status_forcelist=[429, 500, 502, 503, 504], # Коды для повтора
    method_whitelist=["HEAD", "GET", "OPTIONS"] # Методы для повтора
)
adapter = HTTPAdapter(max_retries=retry_strategy)
session.mount("http://", adapter)
session.mount("https://", adapter)
response = session.get("https://example.com")
```

Аутентификация

Basic Authentication

```
from requests.auth import HTTPBasicAuth
# Способ 1
response = requests.get('https://api.example.com/data',
                        auth=HTTPBasicAuth('username', 'password'))
# Способ 2 (сокращенный)
response = requests.get('https://api.example.com/data',
                        auth=('username', 'password'))
```

Token Authentication

```
# Bearer Token
headers = {'Authorization': 'Bearer your_access_token'}
response = requests.get('https://api.example.com/secure', headers=headers)
# API Key
headers = {'X-API-Key': 'your_api_key'}
response = requests.get('https://api.example.com/data', headers=headers)
Digest Authentication
from requests.auth import HTTPDigestAuth

response = requests.get('https://api.example.com/data',
                        auth=HTTPDigestAuth('username', 'password'))
```

Прокси и SSL-сертификаты

Использование прокси

```
proxies = {
    'http': 'http://10.10.1.10:3128',
    'https': 'http://10.10.1.10:1080',
}
response = requests.get('https://example.com', proxies=proxies)
# Прокси с аутентификацией
proxies = {
    'http': 'http://user:pass@10.10.1.10:3128',
    'https': 'http://user:pass@10.10.1.10:1080',
}
```

SSL-сертификаты

```
# Отключение проверки SSL (не рекомендуется в продакшене)
response = requests.get('https://self-signed.badssl.com/', verify=False)
# Указание пути к сертификату
response = requests.get('https://example.com', verify='/path/to/cert.pem')
# Указание сертификата клиента
response = requests.get('https://example.com',
                        cert=('/path/to/client.cert', '/path/to/client.key'))
```

Логирование и отладка запросов

Включение логирования

```
import logging
import requests
# Включение подробного логирования
logging.basicConfig(level=logging.DEBUG)
# Настройка логирования только для requests
requests_log = logging.getLogger("requests.packages.urllib3")
requests_log.setLevel(logging.DEBUG)
requests_log.propagate = True
```

```
response = requests.get('https://example.com')
```

Отладка с помощью hooks

```
def print_request_response(response, *args, **kwargs):
    print(f"Запрос: {response.request.method} {response.request.url}")
    print(f"Ответ: {response.status_code}")
    response = requests.get('https://example.com', hooks={'response':
print_request_response})
```

Работа с потоками данных

Потоковая загрузка

```
response = requests.get('https://example.com/large-file.zip', stream=True)
```

```
# Загрузка по частям
```

```
with open('large-file.zip', 'wb') as f:
```

```
    for chunk in response.iter_content(chunk_size=1024):
```

```
        if chunk:
```

```
            f.write(chunk)
```

```
# Построчная загрузка текстовых файлов
```

```
response = requests.get('https://example.com/data.txt', stream=True)
```

```
for line in response.iter_lines():
```

```
    print(line.decode('utf-8'))
```

Кастомные адаптеры

```
class TimeoutHTTPAdapter(HTTPAdapter):
```

```
    def __init__(self, *args, **kwargs):
```

```
        self.timeout = kwargs.pop("timeout", 5)
```

```
        super().__init__(*args, **kwargs)
```

```
    def send(self, request, **kwargs):
```

```
        kwargs["timeout"] = self.timeout
```

```
        return super().send(request, **kwargs)
```

```
session = requests.Session()
```

```
session.mount("https://", TimeoutHTTPAdapter(timeout=10))
```

Таблица методов и функций библиотеки Requests

Таблица 1 - Методы и функции библиотеки Requests

Метод/Функция	Описание	Пример использования
requests.get()	Выполняет GET-запрос	requests.get('https://api.example.com')
requests.post()	Выполняет POST-запрос	requests.post('https://api.example.com', data={'key': 'value'})
requests.put()	Выполняет PUT-запрос	requests.put('https://api.example.com/1', json={'name': 'New'})
requests.patch()	Выполняет PATCH-запрос	requests.patch('https://api.example.com/1', json={'active': True})
requests.delete()	Выполняет DELETE-запрос	requests.delete('https://api.example.com/1')

requests.head()	Выполняет HEAD-запрос	requests.head('https://api.example.com')
requests.options()	Выполняет OPTIONS-запрос	requests.options('https://api.example.com')
requests.Session()	Создает объект сессии	session = requests.Session()
response.json()	Преобразует ответ в JSON	data = response.json()
response.text	Возвращает текст ответа	content = response.text
response.content	Возвращает бинарное содержимое	binary_data = response.content
response.status_code	Возвращает HTTP статус код	if response.status_code == 200:
response.headers	Возвращает заголовки ответа	content_type = response.headers['Content-Type']
response.cookies	Возвращает куки ответа	session_id = response.cookies['sessionid']
response.url	Возвращает финальный URL	final_url = response.url
response.raise_for_status()	Вызывает исключение для ошибок HTTP	response.raise_for_status()
response.iter_content()	Итерирует по содержимому порциями	for chunk in response.iter_content(1024):
response.iter_lines()	Итерирует по строкам	for line in response.iter_lines():
HTTPBasicAuth()	Базовая HTTP аутентификация	auth=HTTPBasicAuth('user', 'pass')
HTTPDigestAuth()	Digest аутентификация	auth=HTTPDigestAuth('user', 'pass')
HTTPAdapter()	Адаптер для настройки транспорта	adapter = HTTPAdapter(max_retries=3)
Retry()	Конфигурация повторных попыток	retry = Retry(total=3, backoff_factor=1)

Параметры основных методов

Таблица 2 - Параметры основных методов

Параметр	Описание	Применимость
url	URL для запроса	Все методы
params	Параметры URL	Все методы
data	Данные формы	POST, PUT, PATCH
json	JSON данные	POST, PUT, PATCH
headers	Заголовки запроса	Все методы
cookies	Куки	Все методы
files	Файлы для загрузки	POST, PUT, PATCH
proxies	Прокси серверы	Все методы
verify	Проверка SSL	Все методы
cert	Клиентский сертификат	Все методы
stream	Потоковая загрузка	Все методы
allow_redirects	Разрешить редиректы	Все методы

Примеры практического применения

Интеграция с внешними API

Работа с GitHub API

```
def get_user_repos(username):
    url = f'https://api.github.com/users/{username}/repos'
    response = requests.get(url)
    if response.status_code == 200:
        return response.json()
    return None
repos = get_user_repos('octocat')
for repo in repos:
    print(f"{repo['name']}: {repo['description']}")
```

Мониторинг сервисов

```
def check_service_health(url):
    try:
        response = requests.get(url, timeout=5)
        return {
            'status': 'UP' if response.status_code == 200 else 'DOWN',
            'response_time': response.elapsed.total_seconds(),
            'status_code': response.status_code
        }
    except requests.exceptions.RequestException:
        return {'status': 'DOWN', 'response_time': None, 'status_code': None}
health = check_service_health('https://example.com/health')
print(f"Service status: {health['status']}")
```

Отправка уведомлений в Telegram

```
def send_telegram_message(bot_token, chat_id, message):
    url = f'https://api.telegram.org/bot{bot_token}/sendMessage'
    data = {
        'chat_id': chat_id,
        'text': message,
        'parse_mode': 'HTML'
    }
    response = requests.post(url, data=data)
    return response.json()
```

Пример использования

```
send_telegram_message('YOUR_BOT_TOKEN', 'CHAT_ID', 'Привет из Python!')
```

Скрипты автоматизации

```
def download_and_process_data():
    # Загрузка данных
    response = requests.get('https://api.example.com/data.json')
    data = response.json()
    # Обработка данных
    processed_data = [item for item in data if item['active']]
```

```
# Отправка обработанных данных
result = requests.post('https://api.example.com/processed',
                        json=processed_data)
return result.status_code == 200
success = download_and_process_data()
print(f'Обработка {'успешна' if success else 'не удалась'})
```

Оптимизация производительности

Повторное использование соединений

Неэффективно - создает новое соединение для каждого запроса

```
for i in range(10):
```

```
    response = requests.get(f'https://api.example.com/data/{i}')
```

Эффективно - использует пул соединений

```
with requests.Session() as session:
```

```
    for i in range(10):
```

```
        response = session.get(f'https://api.example.com/data/{i}')
```

Асинхронные альтернативы

Для высокопроизводительных приложений рассмотрите использование:

- aiohttp для асинхронных HTTP-запросов;
- httpx как современная альтернатива с поддержкой async/await;
- grequests для псевдо-асинхронных запросов.

Безопасность

Валидация SSL-сертификатов

Всегда проверяйте SSL-сертификаты в продакшене

```
response = requests.get('https://api.example.com', verify=True)
```

Для самоподписанных сертификатов указывайте путь

```
response = requests.get('https://internal-api.company.com',
                        verify='/path/to/ca-bundle.crt')
```

Защита от раскрытия данных

Не логируйте чувствительные данные

```
import logging
```

```
logging.getLogger("requests.packages.urllib3").setLevel(logging.WARNING)
```

Используйте переменные окружения для токенов

```
import os
```

```
api_key = os.getenv('API_KEY')
```

```
headers = {'Authorization': f'Bearer {api_key}'}
```

Пример решения задачи на языке программирования Python

Задача. Напишите программу на языке Python с использованием библиотеки Requests, которая отправляет GET-запрос к указанной веб-странице, получает HTTP-ответ от сервера и выводит на экран код состояния ответа, значение заголовка Content-Type, а также количество символов в полученном содержимом страницы.

Решение задачи:

```
import requests
url = input("Введите URL: ")
response = requests.get(url)
# Код состояния
print("Код состояния:", response.status_code)
# Заголовок Content-Type
content_type = response.headers.get("Content-Type")
print("Content-Type:", content_type)
# Количество символов в ответе
print("Количество символов:", len(response.text))
```

Обмен данными с помощью модуля urllib.request

Модуль **urllib.request** предоставляет расширенные возможности для получения информации из Интернета. Поддерживаются автоматические перенаправления при получении заголовка **Location**, возможность аутентификации, обработка **cookies** и др.

Для выполнения запроса предназначена функция **urlopen()**.

Формат функции:

```
urlopen (<URL-адрес или объект запроса>[, <Данные>][, <Тайм-аут>] [,
cafile=None][, capath=None])
```

В первом параметре задается полный **URL**-адрес или объект, возвращаемый конструктором класса **Request**. Запрос выполняется методом **GET**, если данные не указаны во втором параметре, и методом **POST** в противном случае. При передаче данных методом **POST** автоматически добавляется заголовок **Content-Type** со значением **application/x-www-form-urlencoded**.

Третий параметр задает максимальное время выполнения запроса в секундах. Метод возвращает объект класса **HTTPRequest**.

Этот класс поддерживает следующие методы и атрибуты:

- **read ([<Количество байтов>])** – считывает данные. Если параметр не указан, возвращается содержимое результата от текущей позиции указателя до конца. Если в качестве параметра указать число, то за каждый вызов будет возвращаться указанное количество байтов. Когда достигается конец, метод возвращает пустую строку.

Пример:

```
from urllib.request import urlopen
res = urlopen("http://mail.ru/testrobots.php")
print (res.read(34).decode("cp1251"))
<!DOCTYPE html><html id="nojs" lan
```

– **readline ([<Количество байтов>])** – считывает одну строку при каждом вызове. При достижении конца возвращается пустая строка. Если в необязательном параметре указано число, то считывание будет выполняться до тех пор, пока не встретится символ новой строки (**\n**), символ конца или не будет прочитано указанное количество байтов. Иными словами, если количество символов в строке меньше значения параметра, будет считана одна строка, а не указанное количество байтов. Если количество байтов в строке больше, возвращается указанное количество байтов.

Пример:

```
res = urlopen("http://mail.ru/testrobots.php")
print (res.readline(100).decode("cp1251"))
<!DOCTYPE html><html id="nojs" lang="ru" class=" nonRetina"><head>
<!--MAIL.RU--><meta http-equiv="X-
```

– **readlines ([<Количество байтов>])** – считывает весь результат в список. Каждый элемент списка будет содержать одну строку, включая символ перевода строки. Если параметр задан, считывается указанное количество байтов плюс фрагмент до конца строки. При достижении конца возвращается пустой список.

Пример:

```
res = urlopen("http://mail.ru/testrobots.php")
res.readlines(1)
[<b'<!DOCTYPE html><html id="nojs" lang="ru" class=" nonRetina"><head>
<!--MAIL.RU--><meta http-equiv="X-UA-Compatible"
content="IE=edge"/>&
lt;meta charset="utf-8"/>
<title>Mail.Ru:
. . . Пропуск фрагмента . . .
feed=http://news.mail.ru/rss/&id=5; cycle=1"/><script>\n']
```

– **__next__()** – считывает одну строку при каждом вызове. При достижении конца результата возбуждается исключение **StopIteration**. Благодаря методу **__next__()** можно перебирать результат построчно с помощью цикла **for**. Цикл **for** на каждой итерации будет автоматически вызывать метод **__next__()**.

Пример:

```
res = urlopen("http://mail.ru/testrobots.php")
for line in res: print(line)
```

– **close ()** – закрывает объект результата;

– **geturl ()** – возвращает интернет-адрес полученного документа. Так как все перенаправления автоматически обрабатываются, интернет-адрес полученного документа может не совпадать с адресом, заданным первоначально;

– **info ()** – возвращает объект, с помощью которого можно получить информацию о заголовках ответа сервера. Основные методы и атрибуты этого объекта были рассмотрены при изучении модуля **http.client**.

Пример:

```
res = urlopen("http://mail.ru/testrobots.php")
info = res.info()
info.items()
```

```
[('Server', 'nginx/1.14.1'), ('Date', 'Sun, 06 Jan 2019 07:48:34 GMT'),
('Content-Type', 'text/html; charset=utf-8'), ('Content-Length', '299131'),
('Connection', 'close'),
. . . Пропуск фрагмента . . .
('X-Content-Type-Options', 'nosniff'),
('Strict-Transport-Security', 'max-age=16070400')]
info.get("Content-Type")
text/html; charset=utf-8
info.get_content_type(), info.get_content_charset()
('text/html', 'utf-8')
info.get_content_maintype(), info.get_content_subtype()
('text', 'html')
– code – содержит код возврата в виде числа;
– msg – содержит текстовый статус возврата:
res.code, res.msg
(200, 'OK')
```

Для примера выполним запросы методами GET и POST.

```
from urllib.request import urlopen
from urllib.parse import urlencode
data = urlencode({"color": "Красный", "var": 15}, encoding="cp1251")
# Отправка данных методом GET
url = "http://mail.ru/testrobots.php?" + data
res = urlopen(url)
print(res.read(34).decode("cp1251"))
<!DOCTYPE html><html id="nojs" lan
res.close()
# Отправка данных методом POST
url = "http://mail.ru/testrobots.php"
res = urlopen(url,data.encode("cp1251"))
print(res.read(34).decode("cp1251"))
<!DOCTYPE html><html id="nojs" lan
res.close()
```

Если необходимо задать свое название робота и передать дополнительные заголовки, то следует создать экземпляр класса **Request** и передать его в функцию **urlopen()** вместо интернет-адреса. Конструктор класса **Request** имеет следующий формат:

```
Request (<URL-адрес> [, data=None][, headers={}][, origin_req_host=None][,
unverifiable=False][, method=None])
```

В первом параметре указывается **URL**-адрес. Запрос выполняется методом **GET**, если данные не указаны во втором параметре, или методом **POST** в противном случае. При передаче данных методом **POST** автоматически добавляется заголовок **Content-Type** со значением **application/x-www-form-urlencoded**. Третий параметр задает заголовки запроса в виде словаря. Четвертый и пятый параметр используются для обработки **cookies**. Шестой параметр, поддержка которого появилась в **Python 3.3**, указывает метод передачи данных в

виде строки - например: **"GET"** или **"HEAD"**. За дополнительной информацией по этим параметрам обращайтесь к документации. В качестве примера выполним запросы методами **GET** и **POST**.

```
from urllib.request import urlopen, Request
from urllib.parse import urlencode
headers = { "User-Agent": "MySpider/1.0",
"Accept": "text/html, text/plain, application/xml", "Accept-Language": "ru, ru-
RU",
"Accept-Charset": "windows-1251", "Referer": "/index.php" }
data = urlencode ({"color": "Красный", "var" : 15}, encoding ="cp1251")
# Отправка данных методом GET
url = "http://mail.ru/testrobots.php?" + data
request = Request (url, headers=headers)
res = urlopen (request)
print (res.read(34).decode("cp1251"))
<!DOCTYPE html><html id="nojs" lan
# Отправка данных методом POST
url = "http://mail.ru/testrobots.php"
request = Request(url, data.encode("cp1251"), headers=headers)
res = urlopen (request)
print (res.read(34).decode("cp1251"))
<!DOCTYPE html><html id="nojs" lan
res.close()
```

Определение кодировки

Документы в Интернете могут быть представлены в различных кодировках. Чтобы документ был правильно обработан, необходимо знать его кодировку. Определить кодировку можно по заголовку **Content-Type** в заголовках ответа **Web**-сервера:

Content-Type: text/html; charset=utf-8

Кодировку **Web**-страницы можно также определить по значению параметра content тега **<meta>**, расположенного в разделе **HEAD**:

```
<meta http-equiv="Content-Type" content="text/html; charset=windows-1251">
```

Однако часто встречается ситуация, когда кодировка в ответе сервера не совпадает с кодировкой, указанной в теге **<meta>**, или же таковая вообще не указана. Определить кодировку документа в этом случае позволяет библиотека **chardet**. Установить ее можно из репозитория **PyPI** способом, описанным на [228 шаге](#). Открываем командную строку и набираем в ней команду:

```
c:\python34\scripts\pip install chardet
```

Через некоторое время библиотека будет установлена.

Для проверки установки запускаем редактор **IDLE** и в окне **Python Shell** выполняем следующий код:

```
import chardet
```

```
chardet.__version__
'3.0.4'
```

Определить кодировку строки позволяет функция **detect** (<Последовательность байтов>) из модуля **chardet**. В качестве результата она возвращает словарь с тремя элементами. Ключ **encoding** содержит название кодировки, ключ **language** - название языка, а ключ **confidence** - коэффициент точности определения в виде вещественного числа от 0 до 1. Пример определения кодировки приведен ниже.

```
import chardet
chardet.detect(bytes("Строка", "cp1251"))
{'encoding': 'windows-1251', 'language': 'Russian', 'confidence': 0.99}
chardet.detect(bytes("Строка", "utf-8"))
{'encoding': 'utf-8', 'language': '', 'confidence': 0.99}
```

Если файл имеет большой размер, то вместо считывания его целиком в строку и использования функции **detect()** можно воспользоваться классом **UniversalDetector**. В этом случае можно читать файл построчно и передавать текущую строку методу **feed()**. Если определение кодировки прошло успешно, атрибут **done** будет иметь значение **True**. Это условие можно использовать для выхода из цикла. После окончания проверки следует вызвать метод **close()**. Получить результат определения кодировки позволяет атрибут **result**. Очистить результат и подготовить объект к дальнейшему определению кодировки можно с помощью метода **reset()**. Пример использования класса **UniversalDetector** приведен ниже.

```
from chardet.universaldetector import UniversalDetector
ud = UniversalDetector() # Создаем объект
for line in open("file.txt", "rb"):
    ud.feed(line)        # Передаем текущую строку
    if ud.done: break    # Прерываем цикл, если done == True
ud.close()              # Закрываем объект
print(ud.result)        # Выводим результат
Результат работы приложения:
{'confidence': 0.8726928676836571, 'encoding': 'MacCyrillic', 'language':
'Russian'}
```

Как показали тесты, при использовании кодировки **Windows-1251** файл просматривается полностью, и определение кодировки файла, содержащего 6500 строк, занимает почти секунду. Если сменить кодировку файла на **UTF-8** без **BOM**, то время определения увеличивается до 5 секунд.

Пример решения задачи на языке программирования Python

Задача. Напишите программу на Python с использованием модуля **urllib.request**, которая загружает HTML-код веб-страницы по указанному URL-адресу, преобразует полученные данные в строку и выводит следующие результаты: первые 100 символов страницы, количество символов во всём HTML-коде, а также сообщение об успешном получении данных.

Решение задачи:

```
import urllib.request
url = input("Введите URL: ")
response = urllib.request.urlopen(url)
html_bytes = response.read()
html_text = html_bytes.decode("utf-8", errors="ignore")
print("Первые 100 символов:")
print(html_text[:100])
print("\nОбщее количество символов:", len(html_text))
print("\nДанные получены успешно")
```